
Lösungsvorschlag zu Übungsblatt 2

Abgabe: 04.05.2026, 10:00 Uhr (im Digicampus via VIPS: .java-Dateien für Code, .uxf für UML, .pdf für alles andere)

- Dieses Übungsblatt muss im Team abgegeben werden (Einzelabgaben sind nicht erlaubt!).
- Die **Zeitangaben** geben zur Orientierung an, wie viel Zeit für eine Aufgabe später in der Klausur vorgesehen wäre; gehen Sie davon aus, dass Sie zum jetzigen Zeitpunkt wesentlich länger brauchen und die angegebene Zeit erst nach ausreichender Übung erreichen.

* leichte Aufgabe / ** mittelschwere Aufgabe / *** schwere Aufgabe

Aufgabe 5 (*Benutzereingaben, 18 Minuten*)

In den folgenden Teilaufgaben lernen Sie die **Scanner**-Klasse aus dem Paket `java.util` kennen. Schreiben Sie dazu für jede Teilaufgabe eine Programmklasse und benennen diese jeweils nach der Aufgabennummer.

- a) (*, Scanner-Objekt erstellen, 5 Minuten)

Erstellen Sie ein Objekt der Klasse **Scanner** und übergeben Sie den `InputStream System.in` als Parameter an den Konstruktor. Fordern Sie den Benutzer per Kommandozeilenausgabe auf, eine ganze Zahl zwischen 0 und 50 (einschließlich) einzugeben. Lesen Sie dann mit der Objektmethode `nextInt()` eine ganze Zahl ein und geben diese aus. Am Ende des Programms schließen Sie den Scanner mit der Objektmethode `close()`. Was passiert, wenn Sie etwas anderes als eine ganze Zahl eingeben?

Lösung:

```
import java.util.Scanner;

public class UserInputA {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("Please enter a number between 0 and 50:");
        int n = input.nextInt();
        System.out.println("Input was " + n);
        input.close();
    }
}
```

Wird etwas anderes als eine Zahl eingegeben, so wird eine Exception geworfen. Durch Exceptions (Ausnahmen) werden in Java Fehler der Anwendung kommuniziert – sie enthalten oft hilfreiche Details dazu, an welcher Stelle einer Methode die Ausnahme aufgetreten ist. Die Behandlung von Ausnahmen wird in Kapitel 6 näher behandelt.

- b) (*, Eingabe überprüfen, 6 Minuten)

Passen Sie Ihr Programm aus der vorherigen Teilaufgabe so an, dass zuerst mithilfe der `hasNextInt()`-Objektmethode überprüft wird, ob die Benutzereingabe tatsächlich als ganze Zahl interpretierbar ist. Wenn das der Fall ist, wird diese Zahl eingelesen und überprüft, ob sie wie gefordert zwischen 0 und 50 (einschließlich) liegt. Wurde die Zahl korrekt eingegeben, soll das Programm die Zahl ausgeben. Wird etwas anderes als eine ganze Zahl im entsprechenden Wertebereich eingegeben, soll eine entsprechende Fehlermeldung in die Standardfehlerausgabe geschrieben und das Programm beendet werden.

Lösung:

```
import java.util.Scanner;

public class UserInputB {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int n;
        System.out.println("Please enter a number between 0 and 50:");
        if (!input.hasNextInt() || (n = input.nextInt()) < 0 || n > 50) {
            System.err.println("Wrong input, exiting..");
            input.close();
            return;
        } else {
            System.out.println("Input was " + n);
        }
        input.close();
    }
}
```

- c) (**, Kommandozeilenparameter verarbeiten, 7 Minuten)

Ein Scanner kann nicht nur Tastatureingaben, sondern auch Strings verarbeiten. Recherchieren Sie den dafür passenden Konstruktor und erstellen Sie für jeden übergebenen Kommandozeilenparameter ein `Scanner`-Objekt, das diesen verarbeitet. Der Scanner soll so viele aufeinanderfolgende, durch Whitespace getrennte, ganze Zahlen wie möglich aus dem Kommandozeilenparameter lesen und aufsummieren. Wenn keine ganze Zahl mehr gelesen werden kann, geben Sie die erhaltene Summe aus. Für Kommandozeilenparameter, die keine als ganze Zahl interpretierbare Zeichen enthalten, geben Sie 0 aus.

Lösung:

```
import java.util.Scanner;

public class UserInputC {
    public static void main(String[] args) {
        for (int i = 0; i < args.length; i++) {
            Scanner s = new Scanner(args[i]);
            int sum = 0;
            while (s.hasNextInt()) {
                sum += s.nextInt();
            }
            System.out.println(i + "-ter Kommandozeilenparameter: " + sum);
            s.close();
        }
    }
}
```

Aufgabe 6 (Zeitdarstellung in Java, 22 Minuten)

In dieser Aufgabe arbeiten Sie mit Klassen zur Verwaltung von Datums- und Zeitangaben aus dem Paket `java.time`. Erstellen Sie für jede der Teilaufgaben eine eigene Programmklasse und benennen Sie diese nach der Nummer der jeweiligen Aufgabe.

a) (Zeit bis zur Klausur, *, 7 Minuten)

Erstellen Sie ein Programm, das

- ein `LocalDate`-Objekt für das aktuelle Datum erzeugt,
- ein `LocalDate`-Objekt für den vorläufigen Termin der ersten Klausur für die Vorlesung erstellt (den Termin entnehmen Sie dem Foliensatz 1: Organisatorisches im Digicampus),
- ein `Period`-Objekt erstellt, das die Zeitspanne vom aktuellen Datum zum Prüfungstermin enthält und
- die verbleibenden Monate und Tage auf der Kommandozeile ausgibt.

Lösung:

```
import java.time.LocalDate;
import java.time.Period;

public class ZeitangabenA {
    public static void main(String[] args) {
        LocalDate now = LocalDate.now();
        LocalDate exam = LocalDate.of(2026, 7, 28);
        Period difference = now.until(exam);
        System.out.println("Bis zur Klausur verbleiben " + difference.getMonths() +
            " Monate und " + difference.getDays() + " Tage.");
    }
}
```

b) (Weitere `LocalDate`-Methoden, **, 10 Minuten)

Erstellen Sie ein Programm, das zuerst die Anzahl der Tage bis zum Monatsende bestimmt und ausgibt. Dann soll das Programm zählen, wie viele Samstage und Sonntage noch in diesem Monat verbleiben. Verwenden Sie für beide Anforderungen geeignete Methoden der Klasse `LocalDate` und recherchieren Sie in der API eine Möglichkeit, den Wochentag eines Datums zu bestimmen und zu überprüfen.

Lösung:

```
import java.time.DayOfWeek;
import java.time.LocalDate;

public class ZeitangabenB {
    public static void main(String[] args) {
        LocalDate currentDay = LocalDate.now();
        System.out.println("Tage bis zum Monatsende: " + (currentDay.lengthOfMonth() -
            ↪ currentDay.getDayOfMonth()));
        int sum = 0;
        int currentMonth = currentDay.getMonthValue();
        while (currentDay.getMonthValue() == currentMonth) {
            if (currentDay.getDayOfWeek() == DayOfWeek.SATURDAY || currentDay.getDayOfWeek()
                ↪ == DayOfWeek.SUNDAY) {
                sum += 1;
            }
            currentDay = currentDay.plusDays(1);
        }

        System.out.println("Im Monat verbleiben " + sum + " Wochenendtage");
    }
}
```

c) (*Uhrzeit würfeln, ***, 13 Minuten*)

Erstellen Sie zuerst eine Klassenmethode, die ein `LocalTime`-Objekt mit gültiger, zufälliger Stunden- und Minutenzahl zurückgibt. Den Kopf der Methode legen Sie geeignet selbst fest. In der `main()`-Methode

- bestimmen Sie zwei zufällige Zeiten und speichern die frühere der beiden Zeiten in der Variable `t1`, die spätere in der Variable `t2`,
- geben beide Zeiten in einer eigenen Zeile auf der Kommandozeile aus,
- überprüfen mit einer geeigneten Objektmethode und `ChronoUnit.MINUTES`, ob zwischen `t1` und `t2` mehr als 15 Minuten liegen, und solange dass der Fall ist
 - bestimmen Sie eine neue zufällige Zeit `temp` und
 - wenn `temp` zwischen `t1` und `t2` liegt,
 - überschreiben Sie `t2` mit `temp`
- geben am Ende die neue Zeiten jeweils in einer eigenen Zeile auf der Kommandozeile aus.

Lösung:

```
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;
import java.util.Random;

public class ZeitangabenC {
    public static void main(String[] args) {
        LocalDateTime t1 = randomTime();
        LocalDateTime temp = randomTime();
        LocalDateTime t2;
        if (temp.isBefore(t1)) {
            t2 = t1;
            t1 = temp;
        } else {
            t2 = temp;
        }
        System.out.println("t1: " + t1);
        System.out.println("t2: " + t2);
        while (t1.until(t2, ChronoUnit.MINUTES) > 15) {
            temp = randomTime();
            if (t1.isBefore(temp) && t2.isAfter(temp)) {
                t2 = temp;
            }
        }
        System.out.println("t1: " + t1);
        System.out.println("t2: " + t2);
    }

    public static LocalDateTime randomTime() {
        Random rng = new Random();
        return LocalDateTime.of(rng.nextInt(24), rng.nextInt(60));
    }
}
```

Aufgabe 7 (Klassenkarte implementieren, 25 Minuten)

a) (*, Klassenkarte implementieren, 10 Minuten)

Betrachten Sie die folgende Klassenkarte und die zugehörige Systembeschreibung:

SodaMachine
drinkName count pricePerDrink
getDrinkName getCount setCount getPricePerDrink sellDrink calculateTotalValue

Ein Getränkeautomat verwaltet ein einzelnes Getränk mit einem Namen, einem aktuellen Lagerbestand und einem Preis pro Einheit. Wird ein Getränk verkauft, soll sich der Lagerbestand entsprechend reduzieren. Ein Verkauf ist nur möglich, wenn noch genügend Bestand vorhanden ist. Die Methode zum Verkaufen soll zurückgeben, ob der Verkauf erfolgreich war oder nicht. Zusätzlich soll es eine Methode geben, die den Gesamtwert aller noch vorhandenen Getränke im Automaten berechnet.

Implementieren Sie eine passende Klasse und testen Sie diese mit dem vorgegebenen Hauptprogramm. Vergessen Sie nicht einen passenden Konstruktor, um Objekte der Klasse anlegen zu können.

Lösung:

```
public class SodaMachine {
    private String drinkName;
    private int count;
    private double pricePerDrink;

    public SodaMachine(String drinkName, int count, double pricePerDrink) {
        setCount(count);
        /* Auf die Setter fuer drinkName und pricePerDrink wird hier verzichtet,
           um die Loesung uebersichtlich zu halten. */
        this.drinkName = drinkName;
        this.pricePerDrink = pricePerDrink;
    }

    public String getDrinkName() {
        return drinkName;
    }

    public int getCount() {
        return count;
    }

    public void setCount(int count) {
        this.count = count;
    }

    public double getPricePerDrink() {
        return pricePerDrink;
    }
}
```

```

    public double calculateTotalValue() {
        return count * pricePerDrink;
    }

    public boolean sellDrink() {
        if (count > 0) {
            count--;
            return true;
        } else {
            return false;
        }
    }
}

```

b) (*, Klassenkarte implementieren, 10 Minuten)

Betrachten Sie die folgende Klassenkarte und die zugehörige Systembeschreibung:

MovieTicket
<u>BASE_PRICE</u> movieName startTime
getMovieName setMovieName getStartTime setStartTime calculatePrice getBasePrice

Eine Kinokarte hat einen Filmmamen, eine Startuhrzeit und einen Preis. Es existiert ein fester Basispreis von 8.00\$, der für alle Tickets gleich ist. Es sollen Methoden zur Verfügung gestellt werden, um alle Attribute auszulesen und zu verändern (außer dem Basispreis). Der Basispreis darf nicht verändert werden, soll aber ausgelesen werden können. Zusätzlich soll eine Methode implementiert werden, die den tatsächlichen Ticketpreis berechnet: Für Vorstellungen vor 17:00 Uhr entspricht der Preis dem Basispreis. Zwischen 17:00 Uhr und 22:00 Uhr beträgt der Preis das 1.5-fache des Basispreises. Danach beträgt der Preis das Doppelte des Basispreises.

Implementieren Sie eine passende Klasse. Vergessen Sie nicht einen passenden Konstruktor, um Objekte der Klasse anlegen zu können.

Lösung:

```

import java.time.LocalDateTime;

public class MovieTicket {
    private final static double BASE_PRICE = 8;
    private String movieName;
    private LocalDateTime startTime;

    public MovieTicket(String movieName, LocalDateTime startTime) {
        setMovieName(movieName);
        setStartTime(startTime);
    }

    public String getMovieName() {
        return movieName;
    }
}

```

```

    public void setMovieName(String movieName) {
        this.movieName = movieName;
    }

    public LocalTime getStartTime() {
        return startTime;
    }

    public void setStartTime(LocalTime startTime) {
        this.startTime = startTime;
    }

    public double calculatePrice() {
        if (startTime.isBefore(LocalTime.of(17, 0))) {
            return BASE_PRICE;
        } else if (startTime.isBefore(LocalTime.of(22, 0))) {
            return 1.5 * BASE_PRICE;
        } else {
            return 2 * BASE_PRICE;
        }
    }

    public static double getBasePrice() {
        return BASE_PRICE;
    }
}

```

c) (*, *main()*-Methode implementieren, 5 Minuten)

Implementieren Sie eine Programmklasse, um ihre Implementierung aus der vorherigen Teilaufgabe zu testen. Ihre *main*-Methode soll

- drei Tickets für die Filme
 - „Das große Krabbeln“ um 14:00 Uhr,
 - „Krieg der Sterne“ um 18:00 Uhr und
 - „Der Weiße Hai“ um 23:00 Uhr
 anlegen,
- alle drei Tickets in einem Array passender Länge speichern und
- mithilfe des Arrays den Gesamtpreis aller drei Tickets zusammen bestimmen und ausgeben.

Lösung:

```

import java.time.LocalTime;

public class MovieTheater {
    public static void main(String[] args) {
        MovieTicket m1 = new MovieTicket("Das große Krabbeln", LocalTime.of(14, 0));
        MovieTicket m2 = new MovieTicket("Krieg der Sterne", LocalTime.of(18, 0));
        MovieTicket m3 = new MovieTicket("Der weiße Hai", LocalTime.of(23, 0));
        MovieTicket[] tickets = new MovieTicket[3];
        tickets[0] = m1;
        tickets[1] = m2;
        tickets[2] = m3;
        /*
        alternative One-Liner Loesung:
        MovieTicket[] tickets = {m1, m2, m3};
        */
        double total = 0;
        for (int i = 0; i < tickets.length; i++) {
            total += tickets[i].calculatePrice();
        }
        System.out.println("Gesamtpreis: " + total + "$");
    }
}

```

Aufgabe 8 (Eigene Klasse implementieren, 34 Minuten)

In dieser Aufgabe sollen Sie einfache Klassen zum Verwalten von Wanderstrecken und Wanderern implementieren:

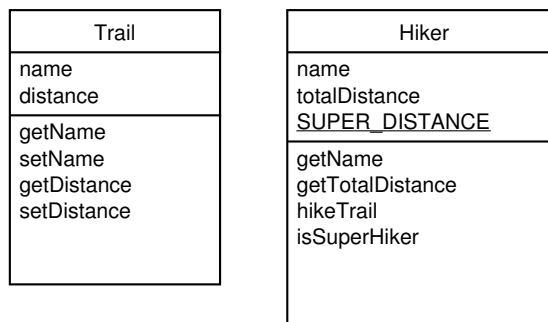
Eine Wanderstrecke hat einen Namen sowie eine Länge in Kilometern. Sie stellt Methoden zum Bearbeiten und Lesen der beiden Werte bereit. Ein Wanderer hat einen Namen und eine Gesamtstrecke, die er bereits gewandert ist. Ein Wanderer stellt eine Methode bereit, um seinen Namen zu lesen. Außerdem kann ein Wanderer eine Wanderstrecke wandern. Dann erhöht sich seine Gesamtstrecke um die Länge der Wanderstrecke. Für Wanderer gibt es den Status Super Wanderer, abhängig von ihrer gewanderten Gesamtstrecke: Ist ein Wanderer mehr als 10 km gewandert, so gilt er als Super Wanderer. Ein Wanderer bietet eine Methode an, um zu überprüfen, ob er ein Super Wanderer ist.

a) (*, Klassenkarte entwerfen, 8 Minuten)

Entwerfen Sie eine Klassenkarte für Wanderstrecken und Wanderer.

Hinweis: auch wenn die Lösungsvorschläge auf Englisch gehalten sind, ist es für Sie vollkommen in Ordnung, sowohl Übungsblätter als auch die Klausur auf Deutsch zu bearbeiten. Bereits während des Studiums werden Sie regelmäßig zu Fragen recherchieren, deren Antworten Sie zum Teil nur auf Englisch finden werden. Sehen Sie dementsprechend die englischsprachigen Lösungsvorschläge als Vorbereitung darauf.

Lösung:



b) (*, Klasse implementieren, 7 Minuten)

Implementieren Sie eine Klasse **Trail** gemäß Ihrer Klassenkarte. Vergessen Sie nicht einen Konstruktor, um neue Objekte vom Typ **Trail** erstellen zu können.

Lösung:

```
public class Trail {
    private String name;
    private double distance;

    public Trail(String name, double distance) {
        setName(name);
        setDistance(distance);
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getDistance() {
        return distance;
    }
}
```

```
    public void setDistance(double distance) {
        this.distance = distance;
    }
}
```

c) (**, Klasse implementieren, 9 Minuten)

Implementieren Sie eine Klasse `Hiker` gemäß Ihrer Klassenkarte. Vergessen Sie nicht einen Konstruktor, um neue Objekte vom Typ `Hiker` erstellen zu können.

Lösung:

```
public class Hiker {
    private String name;
    private double totalDistance;
    private static final double SUPER_DISTANCE = 10;

    public Hiker(String name) {
        /* Weil Setter nicht Teil der Aufgabenstellung waren, ist hier direktes Setzen der
        ↪ Attribute auch erlaubt */
        this.name = name;
        this.totalDistance = 0;
    }

    public String getName() {
        return name;
    }

    public double getTotalDistance() {
        return totalDistance;
    }

    public void hikeTrail(Trail t) {
        totalDistance += t.getDistance();
    }

    public boolean isSuperHiker() {
        return this.totalDistance >= Hiker.SUPER_DISTANCE;
    }
}
```

d) (**, Programmklasse implementieren, 10 Minuten)

Implementieren Sie eine Programmklasse, die

- drei Wanderungen erzeugt:
 - „Stadtspaziergang zu den Mozarts“ mit Länge 2.5 km
 - „Wasser an der Stadtmauer“ mit Länge 4 km
 - „Wertach Vital“ mit Länge 12 km
- zwei Wanderer „Stefan“ und „Aida“ erzeugt,
- Stefan die Wanderungen „Stadtspaziergang zu den Mozarts“ und „Wasser an der Stadtmauer“ wandern lässt,
- die Länge von „Wasser an der Stadtmauer“ auf 6 km setzt,
- Aida die Wanderungen „Wasser an der Stadtmauer“ und „Wertach Vital“ wandern lässt und
- für beide Wanderer den Namen und die insgesamt gewanderte Distanz sowie zusätzlich ausgibt, ob sie jeweils ein Super Wanderer sind. Die Ausgabe soll wie folgt formatiert sein:
[Hiker] has hiked [total distance] km and [is / is not] a Super Hiker!

Lösung:

```
public class HikerMain {
    public static void main(String[] args) {
        Trail t1 = new Trail("Stadtspaziergang zu den Mozarts", 2.5);
        Trail t2 = new Trail("Wasser an der Stadtmauer", 4);
        Trail t3 = new Trail("Wertach Vital", 12);

        Hiker p1 = new Hiker("Stefan");
        Hiker p2 = new Hiker("Aida");

        p1.hikeTrail(t1);
        p1.hikeTrail(t2);
        t2.setDistance(6);
        p2.hikeTrail(t2);
        p2.hikeTrail(t3);

        System.out.print(p1.getName() + " has hiked " + p1.getTotalDistance() + " km and
        ↪ is");
        if (!p1.isSuperHiker())
            System.out.print(" not");
        System.out.println(" a Super Hiker!");

        System.out.print(p2.getName() + " has hiked " + p2.getTotalDistance() + " km and
        ↪ is");
        if (!p2.isSuperHiker())
            System.out.print(" not");
        System.out.println(" a Super Hiker!");
    }
}
```